

Principled Approaches to Score Matching in Diffusion Models

Jonathan Kelly

Supervised by: Prof. Georg Maierhofer and Prof. Eugene Wong

December 2025

Abstract

Diffusion models are a central tool in generative AI, with particularly strong applications in image generation. They rely on a forward stochastic process that perturbs data and a reverse process that reconstructs it. The main challenge is to approximate the score function governing the reverse dynamics accurately enough to produce high-quality samples. In this paper, I will present alternative approaches based on Markovian Denoising Estimation, where the density-inducing process is approximated from samples of the forward process rather than learned by a standard neural network model. We will also explore its qualitative and quantitative performance on some example datasets.

Contents

1	Introduction	2
2	Diffusion Models	2
2.1	Forward Process	3
2.2	Reverse Process	3
2.3	Score Function	4
3	Neural Networks for Score Approximation	4
3.1	Continuous-Time Score Matching	4
3.2	DDPM Noise Prediction	4
3.3	Benefits and Drawbacks of Neural Networks for Score Estimation	6
4	Markovian De-noising Estimation (MDE) for Score Approximation	7
4.1	Motivation	7
4.2	Markovian De-noising Estimation (MDE) and Information Propagation	7
4.3	Approximation of the Score Function	8
4.4	Connection to Nadaraya-Watson Estimation	10
4.5	Data Churning and the Forward Equation	10
5	Evaluation Metrics	11
5.1	Qualitative Assessment	11
5.2	Maximum Mean Discrepancy (MMD)	11
5.3	Wasserstein Distance	12
5.4	Classifier Two-Sample Test (C2ST)	13
5.5	Computational Efficiency	14

6 Experiments and Results	15
6.1 Moons Dataset	15
6.2 Gaussian Mixture Model Dataset	18
6.3 Componentwise Denoising Test	20
7 Conclusion and Future Work	22

1 Introduction

Generative AI aims to learn probability distributions from data well enough to produce new samples with coherent structure. Among modern generative methods, diffusion models have become especially influential because they achieve high-quality generation across images, audio, and many other data modalities. In the score-based formulation of a diffusion model, two stochastic processes are used for the generation process. The first process is a forward stochastic differential equation (SDE) that gradually perturbs data into a simple reference distribution. The second is a reverse-time SDE that transports noise back towards the data distribution. The reverse dynamics are governed by a time-dependent score function, so the practical success of the method depends critically on how accurately that score can be approximated [2].

From a mathematical perspective, diffusion models are attractive because they combine stochastic processes, conditional expectation, and high-dimensional function approximation in one framework. At the same time, they are computationally demanding. In most modern implementations, the score is represented by a neural network that must be trained over many noise levels and then evaluated repeatedly during sampling. Recent survey work emphasises both the breadth of applications of diffusion models and the need for more principled theoretical understanding of score approximation, sampling, and efficiency [1]. This tension between empirical power and computational cost motivates the present essay.

The goal of this project is to study an alternative route to score computation that does not begin with a large neural network. Following the program proposed by Eugene Wong and Georg Maierhofer, we instead investigate whether the score can be approximated directly from sampled forward trajectories and suitable conditional density estimates. This leads to the method of **Markovian De-noising Estimation (MDE)**, which replaces learned score networks by an explicit construction based on propagated sample information.

In this paper, we will give a brief overview of a standard score-based diffusion model that uses a neural network, and then present the MDE framework. We will then implement these approaches on example data sets and evaluate their performance.

2 Diffusion Models

In the continuous-time score-based framework, a diffusion model is specified by a forward stochastic differential equation and a corresponding reverse-time equation. The forward process gradually transports data to a simple reference distribution, while the reverse process transports noise back toward the data distribution [2]. This formulation is the natural starting point for the present essay, since the proposed MDE method also begins from the forward SDE and asks how the reverse drift may be approximated without training a large neural network.

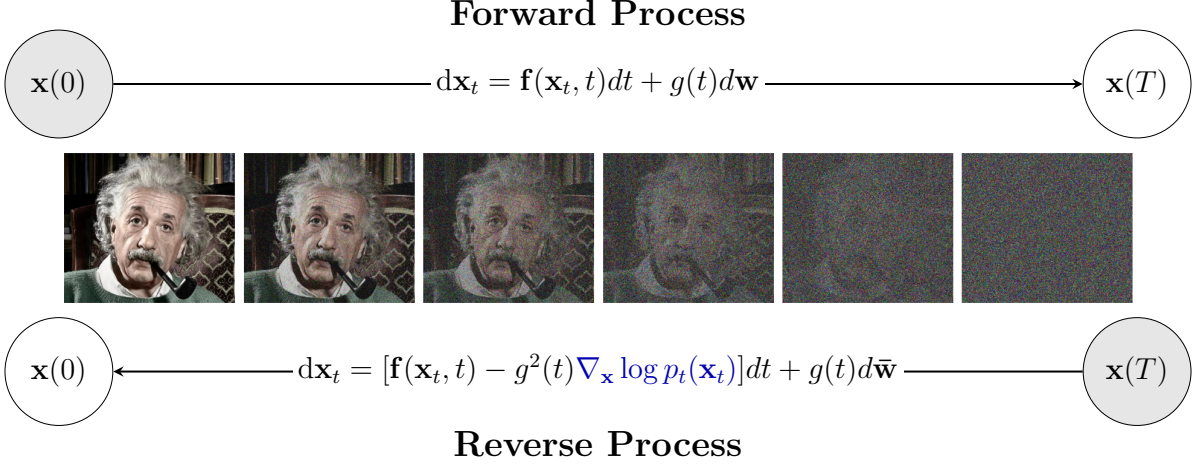


Figure 1: SDE-based formulation of image diffusion.

2.1 Forward Process

The forward process starts from data and transports it toward a simple reference distribution, typically Gaussian noise. In the notation used throughout this essay, the forward dynamics are

$$dX_t = \alpha(X_t, t)dt + \beta(X_t, t)dW_t .$$

Given a finite collection of data points $\{a_k\}$, we regard them as initial conditions and generate the sample paths $\{X_t(a_k)\}$. As t increases, the law of X_t becomes progressively smoother and easier to sample from. The practical role of the forward process is therefore to connect a complicated data distribution to a tractable noise distribution through a continuum of intermediate marginals [2, 1].



Figure 2: Forward diffusion on MNIST dataset

2.2 Reverse Process

Generation is performed by reversing the forward mechanism. Starting from a noise sample, one seeks a reverse process whose terminal distribution matches the data law. In the notation used later in the essay, we write the reverse dynamics as

$$dX_t^- = m(X_t^-, t)dt + \beta(X_t^-, T - t)dW_t^- .$$

The function $m(\cdot, t)$ is the drift of the reverse process. In score-based diffusion models, this function determines how samples should move locally in order to convert noise back into coherent structure. The mathematical problem is therefore to identify $m(\cdot, t)$ accurately enough that the reverse-time simulation genuinely reconstructs the data distribution rather than merely transporting noise.

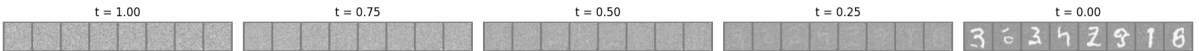


Figure 3: Reverse Diffusion on random noise

2.3 Score Function

The reverse drift is the central object required for generation. In score-based diffusion models, this drift is determined by the score function

$$s(x, t) = \nabla_x \log p_t(x),$$

where p_t is the density of the perturbed data distribution at time t . In the standard score-SDE framework of Song et al., if the forward process is written in the form

$$dX_t = f(X_t, t) dt + g(t) dW_t,$$

then the reverse-time SDE is

$$dX_t = [f(X_t, t) - g(t)^2 \nabla_x \log p_t(X_t)] dt + g(t) d\bar{W}_t, \quad t \text{ runs backward},$$

where p_t is the density of the perturbed data distribution at time t [2]. This identity makes the role of the score explicit: the reverse drift is determined by the gradient $\nabla_x \log p_t(x)$, so sample generation reduces to estimating that time-dependent vector field accurately.

3 Neural Networks for Score Approximation

3.1 Continuous-Time Score Matching

In mainstream diffusion models, the score is not computed explicitly from densities. Instead, one introduces a neural network s_θ and trains it to approximate the score field over time. A representative continuous-time objective is

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim U(0, T)} \mathbb{E}_{x(0)} \mathbb{E}_{x(t) | x(0)} \left[\lambda(t) \|s_\theta(x(t), t) - \nabla_{x(t)} \log p_{0t}(x(t) | x(0))\|_2^2 \right]$$

where $p_{0t}(x(t) | x(0))$ is the forward transition density and $\lambda(t)$ is a weighting function. This is a denoising score-matching objective: the network is trained so that, at each noise level, its output matches the score of the perturbed data distribution. Song et al. show that this continuous-time viewpoint unifies earlier score-matching and diffusion constructions while also supporting numerical SDE solvers and related sampling schemes [2].

3.2 DDPM Noise Prediction

A particularly common discrete formulation is the DDPM noise-prediction parametrisation [3]. Rather than outputting the score field directly, the network is trained to predict the Gaussian noise used in the forward corruption step. Let $x_0 \in \mathbb{R}^d$ denote a clean data point and let $t \in \{1, \dots, T\}$ be a discrete noise level. The forward noising process is

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

where

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Equivalently,

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\alpha_t} x_0, (1 - \bar{\alpha}_t) I).$$

The network takes as input the noisy point and the time index, and may be written as

$$\varepsilon_\theta(x_t, t) = f_\theta([x_t, \text{Emb}(t)]),$$

where $\text{Emb}(t)$ is a learned time embedding. Its output is therefore not a score vector directly, but an estimate of the noise variable ε . The training objective is the mean squared error

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, t, \varepsilon} [\|\varepsilon_\theta(x_t, t) - \varepsilon\|_2^2].$$

Under squared loss, the optimal predictor is the conditional expectation

$$\varepsilon_\theta^*(x_t, t) = \mathbb{E}[\varepsilon \mid x_t, t].$$

Thus the neural network is learning the conditional mean of the added Gaussian noise.

This becomes a score estimator because the forward kernel is Gaussian. Indeed,

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1 - \bar{\alpha}_t} = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}}.$$

Taking conditional expectation with respect to the unknown clean sample x_0 given x_t yields

$$\nabla_{x_t} \log q_t(x_t) = -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \mathbb{E}[\varepsilon \mid x_t, t].$$

Therefore, once the network has learned

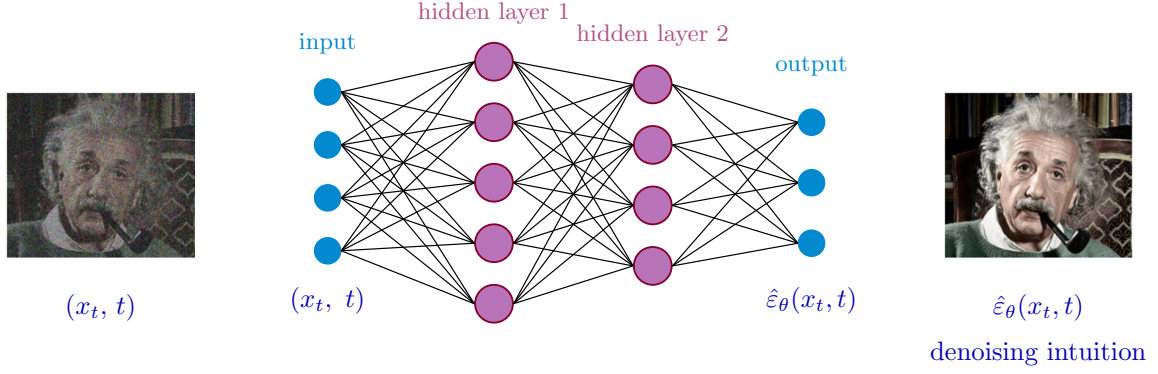
$$\varepsilon_\theta(x_t, t) \approx \mathbb{E}[\varepsilon \mid x_t, t],$$

the score of the noisy marginal is recovered by the explicit rescaling

$$s_\theta(x_t, t) = \nabla_{x_t} \log q_t(x_t) \approx -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}.$$

This is the key mathematical point behind neural noise-prediction models: the score is matched indirectly by first learning the conditional mean of the forward noise.

Figure 4 shows the DDPM-style noise-prediction network used in practice, together with the equivalent denoising intuition conveyed by the Einstein images.



Trained by minimising $\mathbb{E}_{x_0, t, \epsilon} (\|\hat{\epsilon}_\theta(x_t, t) - \epsilon\|^2)$ on the training set

$$\nabla_{x_t} \log q_t(x_t) \approx s_\theta(x_t, t) = -\frac{\hat{\epsilon}_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

Figure 4: DDPM-style neural network for score estimation. The noisy sample x_t together with the time index t enters as input, passes through hidden layers, and the network outputs a prediction of the forward noise $\hat{\epsilon}_\theta(x_t, t)$. The Einstein images indicate the equivalent denoising intuition: noisy data in, cleaner structure out.

The same parametrisation gives the reverse sampling rule used in standard DDPM-style generation. For $t > 1$, one step of the reverse chain is written as

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sqrt{\beta_t} z, \quad z \sim \mathcal{N}(0, I),$$

with the stochastic term removed at the final step. Hence the trained network enters generation through its estimate of the noise, which is analytically equivalent to an estimate of the score.

3.3 Benefits and Drawbacks of Neural Networks for Score Estimation

Neural score models are powerful in practice because they provide flexible high-dimensional parametrisations of the time-dependent score fields used in diffusion models [1, 2]. This flexibility is the main reason they have become the standard tool in modern diffusion modelling: once trained, the same network can be evaluated at many time levels and used to guide the reverse-time sampling procedure. The DDPM and score-SDE frameworks show that this parametric approximation can produce high-quality generative models when the score or equivalent noise-prediction function is learned accurately [3, 2].

The same parametrisation also introduces several drawbacks. First, the model must be trained across many noise levels, so the cost of score estimation is paid through a potentially large optimisation problem. Second, generation still requires repeated network evaluations during the reverse process, making sampling efficiency a central issue in the diffusion-model literature [1]. Third, the learned score is tied to the dataset and training distribution used to fit the parameters. If the target dataset changes, the neural network model is not automatically updated from the new state information; it generally has to be retrained or fine-tuned for the new distribution.

These drawbacks motivate the alternative approach developed next. Instead of treating score estimation purely as a parametric learning problem, the next section asks whether the reverse-time information can be reconstructed more directly from the forward diffusion itself.

4 Markovian De-noising Estimation (MDE) for Score Approximation

Markovian De-noising Estimation (MDE) is the alternative score-computation strategy studied in this essay. The basic idea, developed in Eugene Wong’s notes, is to view the reverse-time drift not as a black-box function to be learned by a large neural network, but as a conditional expectation that can be approximated directly from samples of the forward diffusion [9, 10]. Neural network models amortise reverse-time information into a fixed set of learned parameters; by contrast, MDE aims to reconstruct the reverse drift from sampled forward trajectories and explicit transition-density calculations. In this way, MDE replaces large-scale parametric learning by a combination of path sampling, conditional expectation, and explicit density propagation.

4.1 Motivation

In the reverse-time formulation used in this essay, the reverse drift may be written as the infinitesimal conditional expectation

$$m(y, t) = \lim_{\delta \downarrow 0} \left(\frac{1}{\delta} \right) \mathbb{E} [X_{t+\delta}^- - X_t^- \mid X_t^- = y].$$

This formula is important because it expresses the reverse dynamics as a conditional average of local path increments. Rather than treating m as an unknown function to be learned globally, MDE attempts to estimate this conditional expectation directly from sampled forward trajectories.

Computationally, this offers the prospect of avoiding the training cost of a large score network while retaining a direct connection between the reverse dynamics and the sampled forward paths. Wong’s notes formulate this idea as an alternative to machine learning based on *samples driven computation of score functions* [9]. The aim is to preserve the core diffusion-model framework while replacing the score network by an explicit estimator built from the forward trajectories themselves.

4.2 Markovian De-noising Estimation (MDE) and Information Propagation

The starting point is the forward diffusion

$$dX_t = \alpha(X_t, t) dt + \beta(X_t, t) dW_t, \quad t > 0,$$

and the reverse-time drift

$$m(y, t) = \lim_{\delta \downarrow 0} \left(\frac{1}{\delta} \right) \mathbb{E} [X_{t+\delta}^- - X_t^- \mid X_t^- = y].$$

where $X_t^- = X_{T-t}$ is the time-reversed process [9]. The key observation is that the reverse drift already has the form of a conditional expectation. The MDE programme therefore asks whether this conditional expectation can be approximated directly from sampled forward trajectories instead of through a neural parameterisation.

Let C be a finite collection of initial conditions. For each $a \in C$, solve the forward SDE with initial state $X_0 = a$, using the same underlying Brownian sample path for every run. The resulting trajectories $\{X_t(a)\}_{a \in C}$ are then sampled at equal intervals. Writing

$$\delta = \frac{T}{N}, \quad t_i = T - i\delta, \quad y_i(a) = X_{t_i}^-(a), \quad v_i(a) = y_{i+1}(a) - y_i(a),$$

we obtain the collection of sample pairs

$$\left\{ \left(\begin{bmatrix} t_i \\ y_i(a) \end{bmatrix}, v_i(a) \right) : 0 \leq i \leq N, a \in C \right\}.$$

This sample cloud may be viewed as noisy information about the map $(t, y) \mapsto \mathbb{E}[X_{t+\delta}^- - X_t^- \mid X_t^- = y]$. In Wong's terminology, the problem is thus converted into one of constructing a function from sampled argument-value pairs [9].

The first mechanism proposed in Eugene's paper is *information propagation*. One introduces an auxiliary process Y_t with explicit transition density

$$p_Y(y, t \mid y_i(a), t_i),$$

and uses this density to spread the local information contained in each sample pair through spacetime. This yields the approximation

$$\widehat{\mathbb{E}}[X_{t+\delta}^- - X_t^- \mid X_t^- = y] \approx \frac{\sum_{i,a} v_i(a) p_Y(y, t \mid y_i(a), t_i)}{\sum_{i,a} p_Y(y, t \mid y_i(a), t_i)}.$$

If the auxiliary process has independent Wiener components, then

$$p_Y(y, t \mid y_i(a), t_i) = (2\pi\sigma^2|t - t_i|)^{-n/2} \exp\left(-\frac{\|y - y_i(a)\|^2}{2\sigma^2|t - t_i|}\right),$$

so information propagation reduces to Gaussian kernel smoothing over the sampled paths.

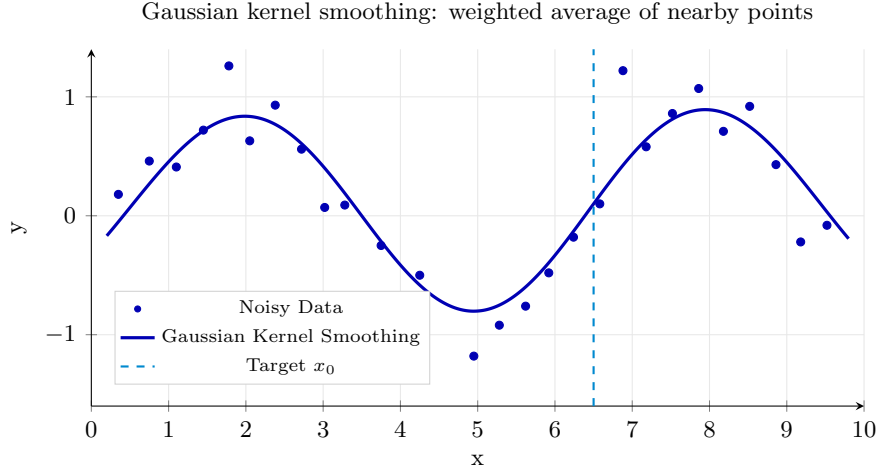


Figure 5: Schematic illustration of Gaussian kernel smoothing as the local averaging mechanism underlying information propagation in MDE. For a query location x_0 , nearby points receive larger Gaussian weights than distant points, and the resulting weighted average produces the smoothed estimate shown by the curve.

4.3 Approximation of the Score Function

The specifically Markovian version of the method estimates the *noise* directly rather than the state increment. Define

$$\widetilde{W}_t^- = W_{T-t}.$$

This is not the Brownian motion driving the true reverse SDE, and it is not adapted to the backward filtration. Its role is purely computational: it allows one to reconstruct the noise term from the sampled forward paths [9]. Discretising the reverse equation gives

$$X_t^-(a) \approx X_{t+\delta}^-(a) + \alpha(X_{t+\delta}^-(a), t + \delta)\delta + \beta(X_{t+\delta}^-(a), t + \delta) \left(\widetilde{W}_t^- - \widetilde{W}_{t+\delta}^- \right).$$

Therefore the corresponding noise increment is approximated by

$$\widetilde{W}_{t_{i+1}}^- - \widetilde{W}_{t_i}^- \approx \beta^{-1}(y_{i+1}(a), t_{i+1}) [v_i(a) + \alpha(y_{i+1}(a), t_{i+1})\delta] = g_i(a).$$

The quantities $g_i(a)$ are sample-based estimates of the local reverse noise needed to reconstruct the reverse drift.

At this point the forward diffusion itself is used as the density-inducing process. Writing

$$p_a^-(y, t \mid y_i(a), t_i)$$

for the conditional density induced by the sampled path beginning at a , one must be careful about time direction. In Doc 2, Wong emphasises that because the score is being reconstructed in reverse time, the relevant conditional density is not always the ordinary forward transition density [10]. If $t > s$, then

$$p_a(y, t \mid x, s) = p(y, t \mid x, s),$$

but if $t < s$, the conditional density must be modified to account for the fixed initial condition a :

$$p_a(y, t \mid x, s) = \frac{p(y, t \mid a, 0)}{p(x, s \mid a, 0)} p(x, s \mid y, t).$$

This is the backward-time density used in the MDE construction. It makes the dependence on the initial sample a explicit while still expressing the result in terms of transition densities of the forward equation. Wong's direct noise-estimation formula then becomes

$$\widehat{\mathbb{E}} [W_{t+\delta}^- - W_t^- \mid X_t^- = y] \approx \frac{\sum_{i,a} g_i(a) p_a^-(y, t \mid y_i(a), t_i)}{\sum_{i,a} p_a^-(y, t \mid y_i(a), t_i)}.$$

Wong further proposes averaging the pathwise estimates $g_i(a)$ over paths:

$$w_i(\delta) = \frac{1}{K} \sum_a g_i(a), \quad p^-(y, t \mid t_i) = \sum_a p_a^-(y, t \mid y_i(a), t_i).$$

This gives the reduced estimator

$$\widehat{\mathbb{E}} [W_{t+\delta}^- - W_t^- \mid X_t^- = y] \approx \frac{\sum_i w_i(\delta) p^-(y, t \mid t_i)}{\sum_i p^-(y, t \mid t_i)}.$$

Substituting this into the reverse drift yields the MDE drift approximation

$$m(y, t) = -\alpha(y, t) + \beta(y, t) \lim_{\delta \downarrow 0} \left\{ \delta^{-1} \widehat{\mathbb{E}} [W_{t+\delta}^- - W_t^- \mid X_t^- = y] \right\}.$$

In this sense, MDE reconstructs the reverse drift by estimating the conditional noise term directly from the forward trajectories. Since the reverse drift is determined by the score in the standard score-SDE formulation, this provides the score information needed for generation.

4.4 Connection to Nadaraya-Watson Estimation

The MDE estimator is closely related to the classical Nadaraya-Watson estimator from nonparametric regression [4]. In its standard form, given observations (X_r, Y_r) , the Nadaraya-Watson estimate of the regression function $m(x) = \mathbb{E}[Y \mid X = x]$ is

$$\hat{m}_{\text{NW}}(x) = \frac{\sum_{r=1}^R K_h(x - X_r) Y_r}{\sum_{r=1}^R K_h(x - X_r)},$$

where K_h is a kernel assigning larger weight to observations close to x . MDE has the same weighted-average structure: the stored noise estimates $g_i(a)$ play the role of the responses Y_r , while the propagated transition density $p_a^-(y, t \mid y_i(a), t_i)$ plays the role of the kernel weight. The main difference is that the MDE weight is not chosen as a generic smoothing kernel; it is induced by the forward diffusion and by the reverse-time conditioning problem. This viewpoint is useful because it explains both the appeal and the limitation of MDE: it is transparent and local, but it inherits the bandwidth, support, and high-dimensional sparsity issues familiar from nonparametric regression.

4.5 Data Churning and the Forward Equation

Doc 2 highlights an important tension in this approach [10]. To make MDE practical, one would like the forward equation to have transition densities that are easy to evaluate. However, if the forward process is too simple, then the generative mechanism becomes too rigid. Wong refers to the ability to generate genuinely new combinations of observed information as *data churning*.

For a purely linear forward equation,

$$dX_t = \alpha(t)X_t dt + \beta(t) dW_t,$$

the transition densities are tractable, but the reverse outputs remain essentially linear combinations of the initial sample set. In Wong’s formulation, this leads to little or no churning. To resolve this, Doc 2 proposes a nonlinear transformation of a tractable linear latent process.

Let Z_t have independent components satisfying the scalar linear diffusions

$$dZ_{it} = \alpha Z_{it} dt + \beta dW_{it}, \quad i = 1, \dots, n,$$

and let ϕ be a scalar nonlinearity applied componentwise, with continuous first and second derivatives, a unique inverse, and nonzero derivative. Define

$$Y_t = \phi(Z_t), \quad X_t = GY_t = G\phi(Z_t),$$

where G is a nonsingular matrix [10]. Since the latent process Z_t is Gaussian, its transition density is explicit and factorises across components. Because X_t is obtained from Z_t by an invertible transformation, the transition density of X_t may still be written explicitly by change of variables, while the nonlinear map ϕ and mixing matrix G introduce the nonlinearity and coordinate interaction needed for data churning.

This mechanism matters because MDE is intended as a generative method, not only as a de-noising method. The nonlinear-transform construction preserves analytic tractability of the forward density while giving the reverse process more expressive power than a purely linear model. The experiments below therefore examine both de-noising accuracy and whether the method exhibits useful churning behaviour in practice.

5 Evaluation Metrics

To assess the proposed method, we use three complementary criteria. First, a *qualitative* criterion asks whether the generated samples resemble the geometry and variability of the target data distribution. Second, several *quantitative* metrics compare the generated and true sample distributions more systematically. Third, *computational efficiency* measures whether MDE reduces the cost of score estimation and sample generation relative to a neural-network baseline. These criteria together separate visual plausibility, statistical fidelity, and runtime cost.

5.1 Qualitative Assessment

Qualitative assessment is the most immediate diagnostic for a generative model. For low-dimensional toy datasets such as the moons dataset or Gaussian mixtures, scatter plots show whether the generated samples reproduce the visible structure of the target law: disconnected components, curvature, concentration, and dispersion. For image-valued data, one instead examines whether generated samples preserve semantically meaningful structure such as digit identity, sharpness, and within-class variability.

Although qualitative assessment is informal, it remains valuable because many generative failures are visually obvious before they are reflected in a single numerical score. A model may approximately match coarse moments while still producing oversmoothed samples, spurious bridges between disconnected components, or collapsed modes. In the experiments below, qualitative inspection is therefore used alongside formal distributional metrics rather than in place of them.

5.2 Maximum Mean Discrepancy (MMD)

Maximum Mean Discrepancy is a kernel-based two-sample statistic designed to test whether two distributions are equal [5]. Given samples $x_1, \dots, x_n \sim P$ and $y_1, \dots, y_m \sim Q$, one embeds both distributions into a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$ with feature map φ and compares their mean embeddings:

$$\text{MMD}(P, Q) = \|\mathbb{E}_{X \sim P}[\varphi(X)] - \mathbb{E}_{Y \sim Q}[\varphi(Y)]\|_{\mathcal{H}}.$$

If the kernel is characteristic, then $\text{MMD}(P, Q) = 0$ if and only if $P = Q$. This makes MMD attractive for generative modelling, since it can detect distributional discrepancies beyond a finite collection of moments. In practice, one computes the squared empirical statistic

$$\widehat{\text{MMD}}^2(P, Q) = \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n k(x_i, x_j) + \frac{1}{m^2} \sum_{i=1}^m \sum_{j=1}^m k(y_i, y_j) - \frac{2}{nm} \sum_{i=1}^n \sum_{j=1}^m k(x_i, y_j),$$

where $k(x, y) = \langle \varphi(x), \varphi(y) \rangle_{\mathcal{H}}$ is the kernel function.

The first two terms measure average within-sample similarity, while the final term measures cross-sample similarity. Smaller values therefore indicate closer agreement between the generated and true distributions. In the present setting, MMD is useful because it is computed directly from samples and does not require explicit density estimation. Its main limitation is that the numerical value depends on the kernel choice and bandwidth, so it is best interpreted comparatively across methods rather than in absolute isolation.

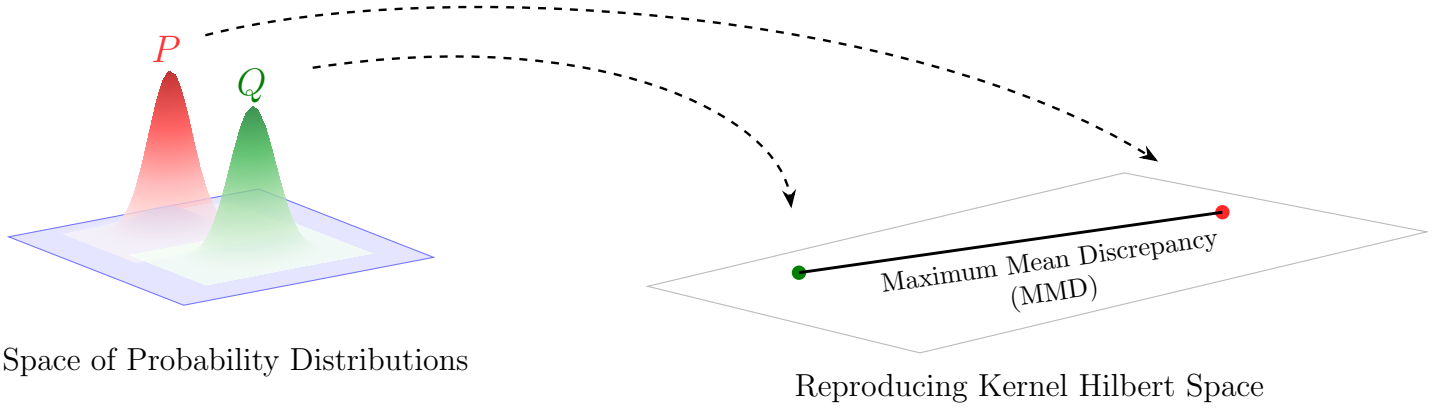


Figure 6: Schematic illustration of two probability distributions P and Q , their embeddings in a reproducing kernel Hilbert space, and the Maximum Mean Discrepancy between those embeddings.

5.3 Wasserstein Distance

The Wasserstein distance provides a geometric notion of discrepancy between probability distributions by asking how much mass must be transported, and at what cost, to transform one distribution into another [6]. For probability measures P and Q on $\mathbb{R}^d$, the p -Wasserstein distance is defined by

$$W_p(P, Q) = \left(\inf_{\gamma \in \Gamma(P, Q)} \int \|x - y\|^p d\gamma(x, y) \right)^{1/p},$$

where $\Gamma(P, Q)$ denotes the set of couplings with marginals P and Q .

For empirical measures with equal sample size,

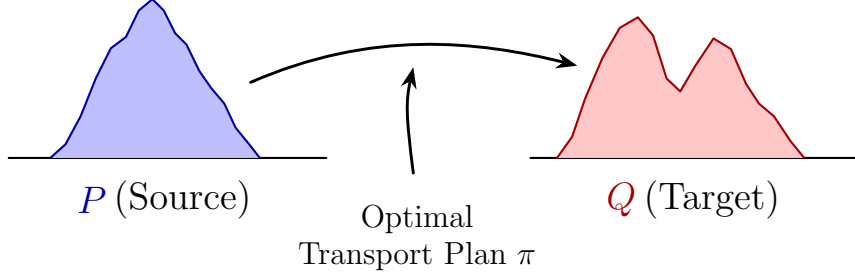
$$P_n = \frac{1}{n} \sum_{i=1}^n \delta_{x_i}, \quad Q_n = \frac{1}{n} \sum_{i=1}^n \delta_{y_i},$$

where δ_{x_i} denotes a unit point mass at the sample location x_i . In this equal-weight empirical setting, the distance becomes an assignment problem:

$$W_p(P_n, Q_n) = \left(\min_{\pi \in S_n} \frac{1}{n} \sum_{i=1}^n \|x_i - y_{\pi(i)}\|^p \right)^{1/p},$$

where S_n is the set of permutations of $\{1, \dots, n\}$.

Unlike MMD, Wasserstein distance depends directly on the geometry of the sample space. This makes it especially informative for low-dimensional datasets such as moons and Gaussian mixtures, where one cares not only about overall overlap of distributions but also about the spatial arrangement of mass. A small Wasserstein distance indicates that generated points lie close to real points in a transport sense, whereas a large value suggests geometric distortion, misplaced mass, or missing modes. Its main drawback is computational: exact optimal transport becomes increasingly expensive as sample size and dimension grow.



$$W_1(P, Q) = \inf_{\pi} \int \|x - y\| d\pi(x, y)$$

Figure 7: Schematic illustration of the Wasserstein-1 distance as an optimal transport problem between a source distribution P and a target distribution Q .

5.4 Classifier Two-Sample Test (C2ST)

The classifier two-sample test converts distribution comparison into a supervised learning problem. Given real samples from P and generated samples from Q , we form the labelled dataset

$$\mathcal{D} = \{(x_i, 1)\}_{i=1}^n \cup \{(y_j, 0)\}_{j=1}^m,$$

and train a binary classifier to distinguish the two classes. If the two distributions are identical, then the labels carry no information and the classifier should perform no better than chance on unseen data. If the distributions differ, then classification accuracy should rise above chance.

In the experiments below, logistic regression is used as the test classifier. For an input z , the model takes the form

$$\mathbb{P}(\ell = 1 \mid z) = \sigma(w^\top z + b), \quad \sigma(u) = \frac{1}{1 + e^{-u}}.$$

The parameters are estimated by minimising binary cross-entropy on a training split:

$$\min_{w, b} - \sum_{i \in \text{train}} [\ell_i \log \hat{p}_i + (1 - \ell_i) \log(1 - \hat{p}_i)].$$

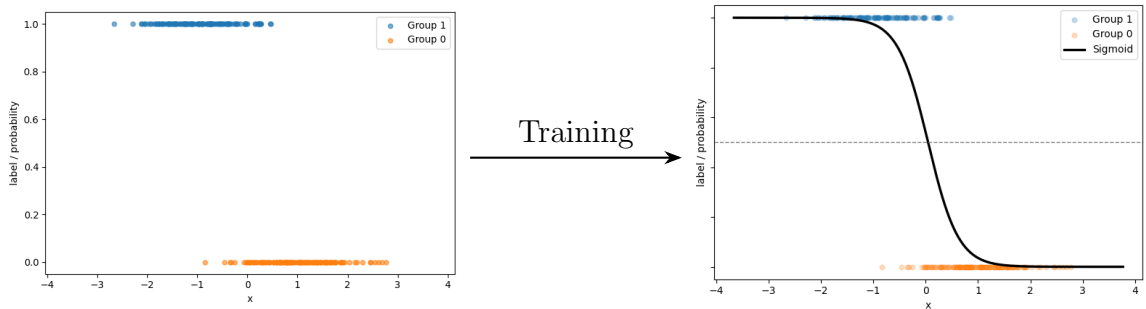


Figure 8: Illustration of the classifier two-sample test before and after training. The classifier is trained to separate the two sample sets, producing the fitted sigmoid decision function.

The resulting statistic is the held-out classification accuracy,

$$\text{C2ST}(P, Q) = \frac{1}{N_{\text{test}}} \sum_{k=1}^{N_{\text{test}}} \mathbf{1}(c(z_k) = \ell_k).$$

Its interpretation is straightforward. If $P = Q$, then one expects $\text{C2ST}(P, Q) \approx 0.5$. Values significantly above 0.5 indicate that the classifier has detected systematic differences between the real and generated samples. Compared with MMD and Wasserstein distance, C2ST has the advantage of adapting automatically to the discrepancies that are easiest to separate in feature space, though its value depends on the chosen classifier family and the train-test split.

5.5 Computational Efficiency

The final criterion is computational efficiency. Since one of the main motivations for MDE is to avoid the training cost of neural score models, it is important to measure not only sample quality but also where the computation is spent. In the experiments, I therefore report wall-clock runtime together with a simple computational-work proxy.

The runtime is decomposed into the main stages used by each method. For the neural baseline, the relevant stages are training and final reverse sampling:

$$T_{\text{NN}} = T_{\text{train}} + T_{\text{sample}}.$$

For MDE, there is no neural-network training phase. Instead, the measured cost is split into forward-bank construction, path and density precomputation, and final reverse sampling:

$$T_{\text{MDE}} = T_{\text{bank}} + T_{\text{precompute}} + T_{\text{sample}}.$$

Alongside wall-clock time, I use a rough work proxy to highlight the dominant cost of each method. For the neural baseline, the main computational burden is training. If P_θ is the number of trainable parameters, B is the batch size, and E is the number of training steps, then

$$W_{\text{NN,train}} \propto P_\theta B E.$$

For MDE, the main computational burden is final reverse sampling, where each generated sample is repeatedly compared against the forward path bank. If d is the data dimension, n is the number of generated samples, N is the number of time steps, and C is the bank size, then

$$W_{\text{MDE,sample}} \propto dnNC.$$

Other timing components, such as neural-network sampling, MDE bank construction, and precomputation, are left to the experiment tables rather than being folded into these simple proxies. The proxies are not exact floating-point operation counts; they are intended to capture the main computational burden of each method.

This criterion must be interpreted together with the statistical metrics above. A method that is fast but produces poor samples is not useful, while a method that is accurate but prohibitively expensive may have limited practical value. The experiments therefore compare sample fidelity, wall-clock time, and work-proxy cost together rather than reducing computational efficiency to a single speedup number.

6 Experiments and Results

Before presenting the results, it is important to note two methodological simplifications used in several experiments. Although the derivation in Doc 2 suggests the path-dependent backward-time density

$$p_a^-(y, t \mid x, s) = \frac{p(y, t \mid a, 0)}{p(x, s \mid a, 0)} p(x, s \mid y, t),$$

some of our implementations instead used the simpler forward transition kernel

$$p_a(y, t \mid x, s) = p(y, t \mid x, s).$$

Empirically, this often produced better de-noising performance. One possible explanation is that the full path-dependent correction introduces a density ratio that can be numerically unstable and can place excessively large weight on a small number of paths. This is analogous to a bias-variance trade-off: the corrected formula is more faithful to the reverse-time conditioning, but the simpler kernel can be more stable in finite-sample regimes.

A second practical simplification is the use of a local i -radius when forming the MDE weighted averages. Instead of summing over every stored time index i , the implementation only uses indices in a small window around the current reverse-time index. This is natural because path states at more distant time indices usually receive smaller transition-density weights and therefore contribute less to the weighted average. The local window keeps the estimator local in time and substantially reduces the cost of reverse sampling.

The experiments are grouped into three tests:

1. **Moons dataset:** a two-dimensional nonparametric test case with visible nonlinear geometry, implemented in `Moons.Experiments.ipynb`.
2. **Gaussian mixture model:** a controlled parametric test case in a higher-dimensional ambient space, implemented in `GMM.Experiments.ipynb`.
3. **Componentwise denoising:** a diagnostic test of whether MDE can be made more scalable by estimating each coordinate separately, implemented in `Componentwise.Denoising.ipynb`.

The full notebooks and supporting code are available in the project repository: <https://github.com/kellyj83/Essay-Repository>.

6.1 Moons Dataset

We begin with the two-moons dataset, since it provides a low-dimensional setting in which both the geometry of the target distribution and the effect of the reverse-time estimator can be inspected directly. The data were standardised before running either method. For the neural baseline, we used a DDPM-style noise-prediction network trained under the usual mean-squared error objective against the forward Gaussian noise. For MDE, we generated a bank of forward trajectories, computed pathwise noise estimates $g_i(a)$, and then reconstructed the reverse drift by local weighted averaging.

The Moons MDE implementation used the two practical approximations mentioned above: the simpler forward transition density and the local i -radius.

The moons experiments were designed to answer three questions:

1. How do the usual sample-quality metrics evolve as reverse denoising proceeds?
2. How does computational effort relate to quality for each method?

3. For MDE, how sensitive is performance to the main tuning parameters, especially the number of reverse steps and the bank size?

To address these questions, the notebook reports MMD, Wasserstein distance, and C2ST both for final samples and at intermediate checkpoints. The efficiency analysis also distinguishes between empirical runtime and a crude proxy for computational work. For the neural network, this proxy is based on parameter-count-scaled training and sampling passes; for MDE, it is based on bank construction together with the number of kernel-style pair evaluations required during reverse sampling.

The moons setting was also used to compare MDE with and without data churning. Since the geometry is visible in two dimensions, this provides a simple way to test whether the nonlinear transformation ϕ and mixing matrix A improve the generated samples beyond merely changing their appearance.

The reported Moons plots and tables use the following parameter settings:

- **Data:** 12,000 two-moons samples with noise 0.05, standardised before training and sampling; a shared subset of 500 samples is used for the comparison.
- **MDE:** bank size $C = 500$, time horizon $T = 1$, time steps $N = 100$, $\delta = 0.01$, $\alpha = -0.5$, $\beta = 0.5$, local i -radius 1, and 500 generated reverse samples.
- **Data churning:** $\phi(z) = z + \frac{1}{3}z^3$, with mixing matrix

$$A = \begin{pmatrix} 1.2 & 0.3 \\ -0.2 & 0.9 \end{pmatrix}.$$

- **Neural baseline:** DDPM-style noise-prediction network with 100 diffusion steps, time embedding dimension 32, hidden width 128, AdamW optimiser with learning rate 2×10^{-3} and weight decay 10^{-4} , trained for 3600 iterations with batch size 500.

The moons experiments support five main conclusions:

1. **MDE performs competitively in the low-dimensional moons setting, but remains sensitive to local sampling errors.** For the parameter setup tested, MDE produced samples of comparable quality to the neural baseline while using a smaller computational-work proxy and a similar wall-clock time. However, some generated points were pushed away from the target moons geometry during denoising, which reduced the quality of the final sample distribution.
2. **Both methods improve during reverse denoising.** In the checkpoint curves, MMD and Wasserstein typically decrease over the reverse simulation. C2ST fluctuates around the ideal value of 0.5, suggesting that neither method is easily distinguishable from the target distribution at later reverse steps.
3. **The computational profiles are different.** The neural method pays most of its cost during training, through repeated forward and backward passes through the network. MDE has essentially no training phase in the usual machine-learning sense, but its cost appears during reverse sampling, where each generated sample is compared against the forward path bank.
4. **MDE is sensitive to its main parameters.** Increasing the bank size or the number of time steps can improve the final denoising result when both parameters are small. However, increasing these parameters also increases computational time.
5. **Data churning improves the geometry.** With the nonlinear transformation included, the metrics are typically improved relative to the non-churning variant, or at least remain competitive

while producing a visibly more convincing shape. This suggests that churning is not merely a visual embellishment, but is tied to improved reverse-time transport.

The corresponding denoising visualisations, checkpoint metrics, and compute summaries are shown below.

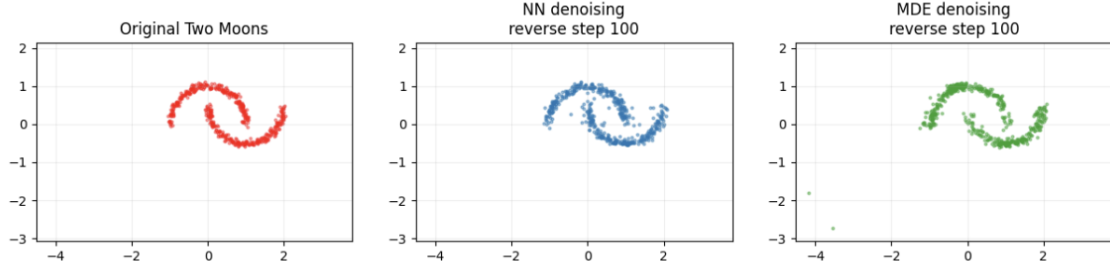


Figure 9: Qualitative denoising snapshots for the Moons experiment. The visualisation shows how samples evolve during the reverse process.

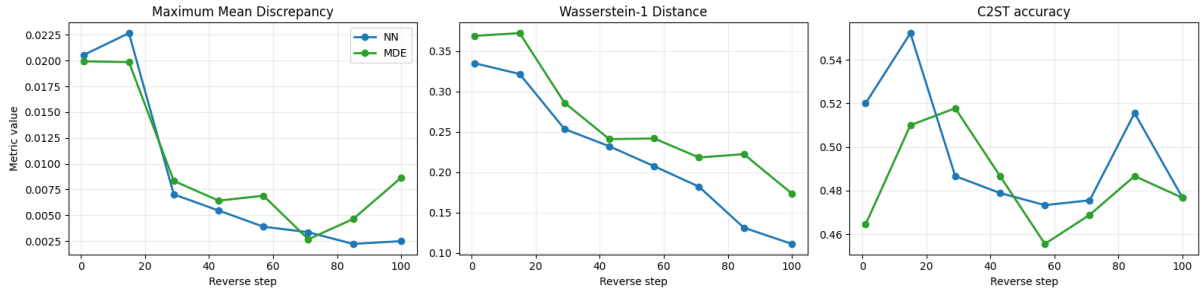


Figure 10: Reverse-process checkpoint diagnostics for the moons experiment. The neural baseline and MDE are evaluated at matching reverse-time checkpoints using MMD, Wasserstein-1 distance, and C2ST accuracy. The figure shows how sample quality evolves during denoising rather than only at the final generated samples.

Reverse step	MMD ²	Wasserstein-1	C2ST accuracy
1	0.020573	0.334803	0.5200
15	0.022668	0.321594	0.5522
29	0.007023	0.253414	0.4867
43	0.005458	0.232007	0.4789
57	0.003889	0.207330	0.4733
71	0.003353	0.181941	0.4756
85	0.002220	0.131159	0.5156
100	0.002485	0.111095	0.4767

Table 1: Neural-network checkpoint metrics for the moons reverse process.

Overall, the moons experiment shows that MDE can reproduce the main reverse-time behaviour of the neural baseline in a low-dimensional setting. Its appeal is that the mechanism is transparent: the generated samples are shaped directly by the forward path bank, local weighting, and reverse-drift reconstruction. The limitation is equally visible: when local support is poor or the smoothing scale is poorly chosen, some samples are pushed away from the moons geometry. This motivates the higher-dimensional Gaussian mixture experiment below.

Reverse step	MMD ²	Wasserstein-1	C2ST accuracy
1	0.019936	0.368878	0.4644
15	0.019860	0.372165	0.5100
29	0.008332	0.285961	0.5178
43	0.006417	0.240850	0.4867
57	0.006895	0.241715	0.4556
71	0.002668	0.218148	0.4689
85	0.004630	0.222289	0.4867
100	0.008645	0.173451	0.4767

Table 2: MDE checkpoint metrics for the moons reverse process.

Quantity	Neural network	MDE
Checkpoint-history sampling time	0.034s	2.860s
Total compute time	3.602s	2.866s
Total computational work proxy	7.584×10^{10}	1.503×10^8
Training time	3.571s	—
Bank construction time	—	0.004s
Precompute time	—	0.002s
Final sampling time	0.031s	2.860s
Training work proxy	7.379×10^{10}	—
Bank construction work proxy	—	1.000×10^5
Precompute work proxy	—	2.000×10^5
Final sampling work proxy	2.050×10^9	1.500×10^8

Table 3: Runtime and computational-work summaries for the moons experiment.

6.2 Gaussian Mixture Model Dataset

The Gaussian mixture experiment extends the comparison beyond the two-dimensional moons setting. Here the target law is a multimodal Gaussian mixture in a higher-dimensional ambient space, so the methods must recover separated regions of high probability without relying on the simple crescent geometry of the moons dataset. In the experiment reported here, the GMM is six-dimensional, with visualisations showing a two-dimensional projection. This setting is useful because it retains a controlled synthetic structure while making the local nonparametric estimation problem more difficult.

As in the moons experiment, the reported GMM implementation used the simpler forward transition density in place of the full path-dependent p_a^- density. It also used a local i -radius when forming the MDE weighted average, so that each reverse step only compares against nearby stored time indices rather than the entire trajectory bank. This approximation was necessary to keep the higher-dimensional experiment computationally manageable.

The reported GMM plots and tables use the following parameter settings:

- **Data:** 12,000 samples from an isotropic Gaussian mixture with 8 components in dimension $d = 6$, standardised before training and sampling; a shared subset of 500 samples is used for the comparison.
- **MDE:** bank size $C = 500$, time horizon $T = 1$, time steps $N = 100$, $\delta = 0.01$, $\alpha = -0.5$, $\beta = 0.5$, kernel scale $\sigma = 0.5$, local i -radius 1, and 500 generated reverse samples.
- **Data churning:** the same componentwise nonlinearity $\phi(z) = z + \frac{1}{3}z^3$, with a deterministic well-conditioned 6×6 mixing matrix.
- **Neural baseline:** DDPM-style noise-prediction network with 100 diffusion steps, time embedding dimension 32, hidden width 128, AdamW optimiser with learning rate 2×10^{-3} and weight decay 10^{-4} , trained for 3600 iterations with batch size 500.

The GMM experiment supports three main conclusions:

1. **The neural baseline remains stable in higher dimension.** Across the reverse process, the neural baseline steadily improves on MMD and Wasserstein distance. C2ST remains close to 0.5, although here the MMD and Wasserstein trends provide the clearest evidence of improvement.
2. **MDE deteriorates during reverse denoising.** In this run, the MDE MMD and Wasserstein values increase over later checkpoints. This suggests that the local weighted estimator is struggling to recover the correct reverse drift in the six-dimensional mixture setting.
3. **The computational burden shifts against MDE.** The neural baseline still pays most of its cost during training, while final sampling remains very fast. MDE spends most of its time in reverse sampling, where each generated sample requires repeated comparisons against the trajectory bank. In this experiment, MDE is therefore both slower in wall-clock time and worse on the final quantitative metrics.

This does not contradict the low-dimensional moons results. Rather, it identifies the regime where the nonparametric nature of MDE becomes a limitation: as dimension increases, local neighbourhoods become sparse, so the kernel-style conditional expectation used by MDE has less reliable support.

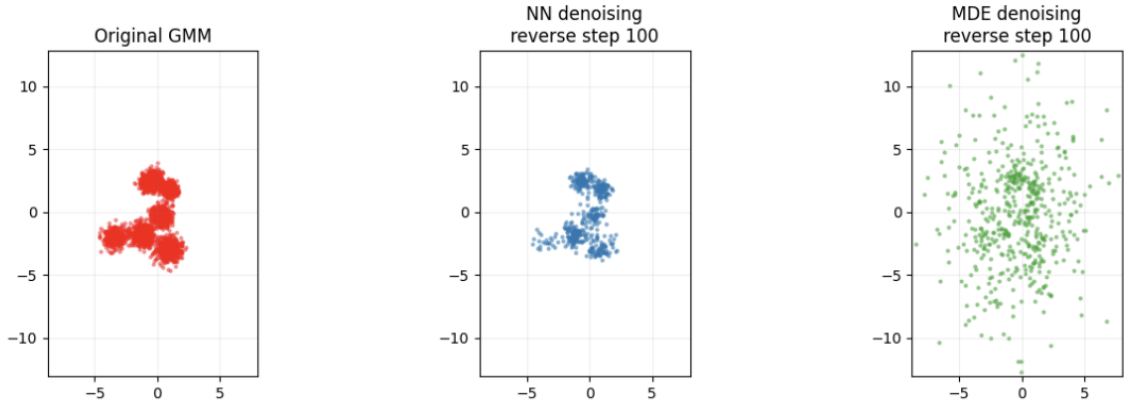


Figure 11: Qualitative denoising snapshots for the Gaussian mixture experiment, shown as a two-dimensional projection of the six-dimensional samples. The visualisation shows how samples evolve during the reverse process and illustrates the increased difficulty of recovering the multimodal GMM structure in the higher-dimensional setting.

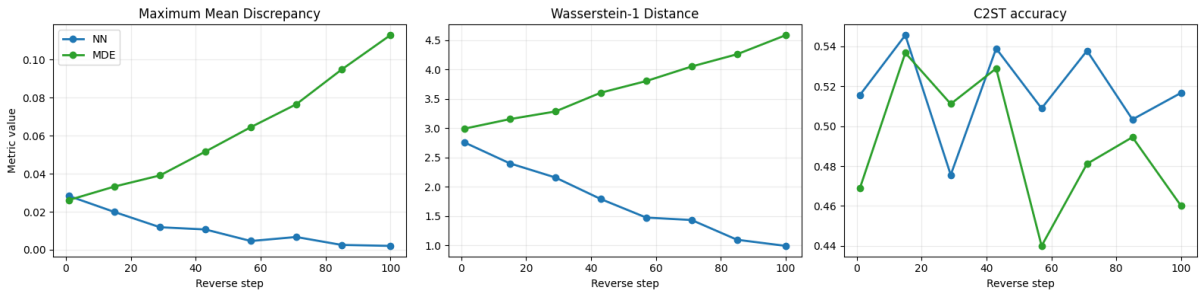


Figure 12: Reverse-process checkpoint diagnostics for the Gaussian mixture experiment. The neural baseline steadily improves across the reverse trajectory, while MDE deteriorates on the MMD and Wasserstein metrics as denoising proceeds.

Reverse step	MMD ²	Wasserstein-1	C2ST accuracy
1	0.028260	2.753508	0.5156
15	0.019857	2.396350	0.5456
29	0.011855	2.156058	0.4756
43	0.010674	1.788290	0.5389
57	0.004598	1.472617	0.5089
71	0.006716	1.430752	0.5378
85	0.002539	1.094336	0.5033
100	0.002033	0.989484	0.5167

Table 4: Neural-network checkpoint metrics for the Gaussian mixture reverse process.

Reverse step	MMD ²	Wasserstein-1	C2ST accuracy
1	0.026189	2.988941	0.4689
15	0.033272	3.151661	0.5367
29	0.039118	3.281770	0.5111
43	0.051663	3.602319	0.5289
57	0.064516	3.800392	0.4400
71	0.076557	4.048985	0.4811
85	0.094843	4.257654	0.4944
100	0.112921	4.583533	0.4600

Table 5: MDE checkpoint metrics for the Gaussian mixture reverse process.

Quantity	Neural network	MDE
Checkpoint-history sampling time	0.038s	7.073s
Total compute time	4.300s	7.307s
Total computational work proxy	7.774×10^{10}	2.705×10^9
Training time	4.268s	—
Bank construction time	—	0.039s
Precompute time	—	0.023s
Final sampling time	0.032s	7.245s
Training work proxy	7.564×10^{10}	—
Bank construction work proxy	—	1.800×10^6
Precompute work proxy	—	3.600×10^6
Final sampling work proxy	2.101×10^9	2.700×10^9

Table 6: Runtime and computational-work summaries for the Gaussian mixture experiment.

6.3 Componentwise Denoising Test

As a further diagnostic, I also tested a componentwise version of the MDE denoising procedure. This test was not intended as a replacement for the joint estimator, but as a way to understand whether coordinatewise simplification could reduce the high-dimensional cost. Instead of forming a full joint density weight, the method treats each coordinate of the state vector as a separate scalar denoising problem and then assembles the reverse drift coordinate by coordinate.

Unlike the moons and GMM experiments above, this test was used to incorporate the corrected p^- directionality from Section 4 rather than the simpler forward-kernel approximation with a local i -radius. In particular, when the conditioning time lies after the query time, the implementation uses the backward-time density correction associated with the fixed initial sample. This made the test a useful diagnostic for whether the more literal MDE formula improves denoising once the estimator is made componentwise.

The reported componentwise denoising plot uses the following parameter settings:

- **Data:** 1000 two-moons samples with noise 0.05, used directly as the forward path bank.
- **Forward process:** time horizon $T = 1$, time steps $N = 75$, $\delta = 1/75$, $\alpha = -0.5$, and $\beta = 0.5$.

- **Componentwise estimator:** no data churning, so $\phi(x) = x$ and $G = I$; corrected one-dimensional p^- weights are used separately for each coordinate.
- **Reverse sampling:** 1000 generated reverse samples, initialised from $\mathcal{N}(0, 0.5^2 I)$, using a one-sided future-time window with i -radius 10.

In the simplified setting without data churning, the observed process is

$$dX_t^j = \alpha X_t^j dt + \beta dW_t^j, \quad j = 1, \dots, n.$$

For a time discretisation $\delta = T/N$, and reverse-time path values

$$y_i(a) = X_{t_i}^-(a), \quad v_i(a) = y_{i+1}(a) - y_i(a),$$

the scalar pathwise noise estimate for component j is

$$g_i^j(a) = \beta^{-1} \left(v_i^j(a) + \alpha y_{i+1}^j(a) \delta \right).$$

For a query point y , the componentwise method then uses the scalar version of the corrected backward-time density. For stored times $t_i > t_k$, this takes the form

$$p_a^{-,j}(y^j, t_k | y_i^j(a), t_i) = \frac{p^j(y^j, t_k | a^j, 0)}{p^j(y_i^j(a), t_i | a^j, 0)} p^j(y_i^j(a), t_i | y^j, t_k),$$

where p^j denotes the one-dimensional Gaussian transition density for component j .

The conditional expectation is estimated separately in each coordinate:

$$\hat{E}^j(y^j, t_k) = \frac{\sum_i \sum_a g_i^j(a) p_a^{-,j}(y^j, t_k | y_i^j(a), t_i)}{\sum_i \sum_a p_a^{-,j}(y^j, t_k | y_i^j(a), t_i)}.$$

The reverse drift is then assembled as

$$m(y, t_k) = (m^1(y^1, t_k), \dots, m^n(y^n, t_k)), \quad m^j(y^j, t_k) = -\alpha y^j + \beta \frac{\hat{E}^j(y^j, t_k)}{\delta}.$$

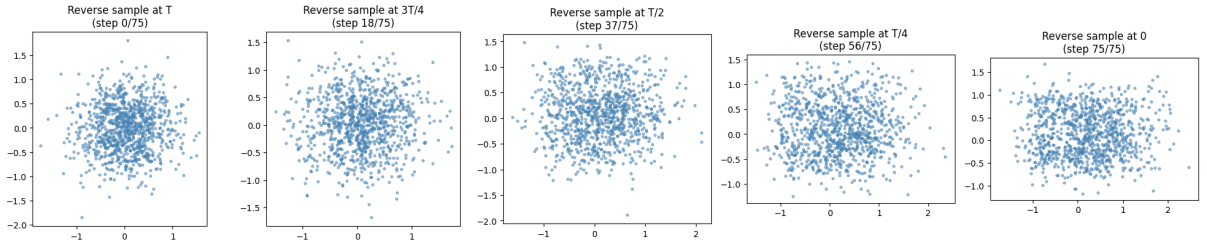


Figure 13: Componentwise denoising on the moons dataset. The qualitative result shows that treating each coordinate as an independent scalar denoising problem does not recover the joint two-dimensional geometry as reliably as the joint estimator.

The componentwise test led to three main findings:

1. **The componentwise method did not recover the joint geometry well.** Even on the two-dimensional moons dataset, treating each coordinate as an independent scalar denoising problem lost important dependence between coordinates. This suggests that the main limitation is structural: the estimator no longer captures the conditional density of the full state vector.

2. **Using the corrected p^- formula was not enough to fix the issue.** Implementing the corrected formula for $p(y, t \mid x, s)$ in the case $s > t$ did not improve the result enough to make componentwise denoising successful. I also observed that, even for the joint denoising method, the corrected p^- formula produced noisier generative results than the simpler forward-kernel approximation. This is consistent with the idea that a theoretically richer weighting rule can become less stable in the finite-sample regime.
3. **Increasing the available samples became computationally expensive.** Using all admissible samples after time t quickly increased the work per reverse step, and the cumulative cost over the full reverse process grew very quickly. Increasing the bank size C , increasing the number of time steps N , and widening the one-sided window all made the computation slow compared with the previous experiments.

To check whether numerical underflow was responsible, I also normalised the weights by dividing p_a^- by its maximum value before forming the weighted sums. This kept more terms active in the sum, and the normalisation masses did not appear to go to zero or become NaN or infinite. Even with this stabilisation, the componentwise method remained qualitatively weaker, reinforcing the conclusion that the loss of joint geometric information is the main limitation.

7 Conclusion and Future Work

This essay investigated Markovian De-noising Estimation as a direct, sample-based alternative to neural score estimation in diffusion models. The central idea was to replace a learned score network with an explicit conditional-expectation estimator constructed from forward diffusion trajectories.

In low-dimensional settings, especially the two-moons experiment, this approach performed well. MDE produced convincing reverse-time samples, gave competitive values for MMD, Wasserstein distance, and C2ST, and avoided the expensive training phase required by the neural baseline. Its computational burden appeared mainly during reverse sampling, while the neural baseline paid most of its cost upfront during optimisation. In this regime, MDE therefore offers a useful form of computational and iterative efficiency: it is transparent, relatively cheap to initialise, and closely tied to the mathematics of the reverse-time drift.

The higher-dimensional Gaussian mixture experiment showed the main limitation. As the dimension increased, the local nonparametric estimator became much less reliable. The neural baseline continued to improve over the reverse process, whereas the MDE metrics deteriorated in later denoising steps. This behaviour is consistent with the curse of dimensionality. Since MDE estimates the reverse drift through local weighted averages over a finite path bank, its effectiveness depends on having enough nearby and relevant samples. In higher dimensions, local neighbourhoods become sparse, transition-density weights become harder to use reliably, and the estimator can lose the support needed to reconstruct the reverse dynamics.

The componentwise denoising experiment clarified this limitation further. Treating each coordinate as an independent scalar denoising problem reduces the difficulty of estimating a full joint density, but it also removes the dependence structure between coordinates. For the moons dataset, the two coordinates together define the geometry of the dataset. Once the estimator ignores the conditional density linking different coordinates, the reverse process loses structural information as it denoises. The componentwise method therefore did not recover the target geometry well.

Overall, the results suggest that MDE is a promising and interpretable alternative to neural score estimation in low-dimensional or well-supported regimes, but that it does not yet scale reliably to higher-dimensional generative modelling. Its strengths are transparency, direct connection to conditional expec-

tation, and reduced training cost. Its weaknesses are local support dependence, sensitivity to smoothing choices, and degradation when the geometry of the data cannot be captured by the available forward path bank.

Future work could focus on improving the stability and scalability of the MDE estimator. One direction is to study better weighting and smoothing rules, such as adaptive bandwidths, local covariance estimates, or regularised density ratios that reduce variance without oversmoothing the reverse drift.

A further direction for future work is suggested by Wong’s more recent research on computing score functions without explicit density formulae. The newer formulation suggests instead estimating the reverse noise term more directly from sampled paths, so that one does not need the full transition density of the forward process. In this view, the goal is to reconstruct the reverse drift from conditional noise information using only simpler local conditional densities or short-time transition approximations. This could reduce one of the main practical weaknesses observed in the MDE experiments: the dependence on explicit density ratios and high-dimensional kernel weights. Investigating this density-free or density-light version of MDE may therefore provide a more scalable route to sample-driven score estimation.

References

- [1] Chen, M., Mei, S., Fan, J., and Wang, M. (2024). An overview of diffusion models: Applications, guided generation, statistical rates and optimization. *arXiv preprint arXiv:2404.07771*.
- [2] Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. (2021). Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*.
- [3] Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 33, 6840–6851.
- [4] Tsybakov, A. B. (2009). *Introduction to Nonparametric Estimation*. Springer Series in Statistics. Springer.
- [5] Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., and Smola, A. (2012). A kernel two-sample test. *Journal of Machine Learning Research*, 13(25), 723–773.
- [6] Kolouri, S., Park, S. R., Thorpe, M., Slepčev, D., and Rohde, G. K. (2017). Optimal mass transport: Signal processing and machine-learning applications. *IEEE Signal Processing Magazine*, 34(4), 43–59.
- [7] Silverman, B. W. (1986). *Density Estimation for Statistics and Data Analysis*. Chapman & Hall/CRC.
- [8] Wong, E. (2025). Conditional expectation and generative AI. In preparation, University of California, Berkeley.
- [9] Wong, E. (2025). Samples driven computation of score functions. Unpublished note.
- [10] Wong, E. (2025). Computing score functions: Focus on the forward equation. Unpublished note.